

**LAPORAN AKHIR PROJECT BASED LEARNING
(PBL)**
Mata Kuliah Algoritma & Struktur Data (TPL2106)
**Tahun Akademik 2025/2026 Dosen : Dr. Shelve Nidya
Neyman**

Manajemen Tumpukan Gudang Barang Berat

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:

Muhammad Zeya Rieqi (J0403251082)

Nelson Arsel Warae (J0403251061)

Muhammad Danish Rahmanu Ghaisan (J0403251124)

Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Puji dan syukur kami panjatkan kehadiran Tuhan Yang Maha Esa atas selesainya proyek Project-Based Learning (PBL) mata kuliah Algoritma & Struktur Data dengan judul “Manajemen Tumpukan Gudang Barang Berat Menggunakan Struktur Data Stack”. Proyek ini disusun sebagai sarana penerapan konsep struktur data, khususnya Stack, ke dalam permasalahan nyata pengelolaan barang gudang.

Kami mengucapkan terima kasih kepada Dr. Shelve Nidya Neyman selaku dosen pengampu mata kuliah Algoritma & Struktur Data yang telah memberikan bimbingan selama proses pembelajaran dan pengerjaan proyek ini. Terima kasih juga kami sampaikan kepada seluruh anggota kelompok yang telah berkontribusi dan bekerja sama dengan baik dari tahap perancangan hingga pengujian program.

Kami menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun implementasi program. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi perbaikan ke depannya. Semoga laporan ini dapat memberikan manfaat dan menjadi referensi yang baik bagi pembaca.

DAFTAR ISI

KATA PENGANTAR	2
DAFTAR ISI	3
BAB I. PENDAHULUAN.....	5
1.1 Latar Belakang	5
1.2 Tujuan Proyek	5
1.3 Ruang Lingkup.....	6
BAB II. ANALISIS DAN PERANCANGAN.....	6
2.1 Deskripsi Sistem	6
2.2 Struktur Data yang Digunakan.....	7
Alasan Pemilihan Struktur Data.....	7
2.3 Algoritma yang Digunakan	8
Searching.....	8
Sorting.....	8
2.4 Diagram Alir Program (Flowchart).....	9
2.5 Struktur Program.....	9
BAB III. IMPLEMENTASI PROGRAM	10
3.1 Tampilan Menu Utama.....	10
3.2 Fitur Create	10
3.3 Fitur Read.....	11
3.4 Fitur Update	11
3.5 Fitur Delete	12
3.6 Fitur Searching.....	12
3.7 Fitur Sorting.....	13
3.8 Penyimpanan Data (File Handling)	14
3.9 Validasi Input dan Error Handling	15
BAB IV. PENGUJIAN DAN ANALISIS	15
4.1 Skenario Pengujian	15
4.2 Kelebihan Sistem	16
4.4 Keterbatasan Sistem.....	16

BAB V. KENDALA DAN SOLUSI	17
BAB VI. PEMBAGIAN TUGAS ANGGOTA	18
BAB VII. KESIMPULAN DAN SARAN	18
Kesimpulan	18
Saran Pengembangan	19
DAFTAR PUSTAKA	20
LAMPIRAN	21
Lampiran A. Link GitHub	21
Lampiran B. Screenshot Program	21
Lampiran C. Struktur Folder Proyek	21
Lampiran D. Commit History GitHub	22
Lampiran E. Source Code Utama	24

BAB I. PENDAHULUAN

1.1 Latar Belakang

Pengelolaan barang di gudang sering kali dilakukan dengan cara menumpuk barang satu di atas barang lainnya, terutama untuk barang-barang berat yang disusun secara vertikal demi efisiensi ruang. Pola penyusunan semacam ini secara alami mengikuti prinsip Last In First Out (LIFO): barang yang paling terakhir diletakkan berada paling atas dan harus diambil lebih dahulu sebelum barang di bawahnya dapat dijangkau. Dalam praktiknya, pengelolaan tumpukan barang secara manual rawan menimbulkan kesalahan, seperti tidak terpantaunya total berat tumpukan yang dapat melebihi batas aman, kesulitan melacak urutan barang yang masuk, serta hilangnya catatan ketika informasi hanya disimpan secara lisan atau pada catatan fisik yang mudah rusak atau hilang.

Untuk mengatasi permasalahan tersebut, proyek ini membangun aplikasi berbasis terminal (Command Line Interface) yang merepresentasikan tumpukan barang gudang menggunakan struktur data Stack berbasis Linked List. Struktur data ini dipilih karena secara konseptual paling mendekati kondisi fisik nyata sebuah tumpukan barang. Aplikasi ini juga dilengkapi dengan mekanisme penyimpanan data permanen menggunakan file teks (.txt), validasi kapasitas berat maksimum, fitur pencarian (searching) untuk menemukan barang dengan nama yang tidak diketahui secara spesifik, serta fitur pengurutan (sorting) untuk membantu pengguna melihat data dari berbagai sudut pandang, seperti berdasarkan berat, urutan masuk, maupun alfabet nama barang.

1.2 Tujuan Proyek

1. Membuat aplikasi berbasis terminal menggunakan bahasa Python untuk mengelola tumpukan barang gudang.
2. Mengimplementasikan struktur data Stack berbasis Linked List sesuai prinsip LIFO yang merepresentasikan kondisi fisik tumpukan barang secara nyata.

3. Menerapkan konsep file handling agar data barang tersimpan secara permanen meskipun program ditutup dan dijalankan kembali.
4. Mengembangkan fitur pencarian (searching) dan pengurutan (sorting) untuk mempermudah pengelolaan data barang dalam jumlah besar.
5. Menambahkan validasi kapasitas berat maksimum gudang untuk mencegah kelebihan muatan yang dapat membahayakan kestabilan tumpukan.

1.3 Ruang Lingkup

- Platform: Console / Command Line Interface (CLI)
- Bahasa pemrograman: Python
- Struktur data utama: Stack (Linked List)
- Penyimpanan data: file TXT (format kustom terpisah tanda “|”)
- Fitur CRUD: tambah barang (push), hapus barang (pop), ubah informasi barang, lihat seluruh barang
- Fitur Searching: pencarian barang berdasarkan kata kunci sebagian nama (partial match)
- Fitur Sorting: tiga mode pengurutan (berat, indeks masuk, dan alfabet), masing-masing ascending dan descending
- Validasi input pengguna dan validasi kapasitas maksimum gudang (100 kg)

BAB II. ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Sistem yang dibangun memungkinkan pengguna untuk menambah barang ke dalam tumpukan (push), mengeluarkan barang dari posisi teratas (pop), melihat seluruh isi tumpukan dalam berbagai urutan tampilan, mencari barang berdasarkan kata kunci nama yang tidak spesifik, mengubah informasi barang berdasarkan indeks unik, serta memantau jumlah dan total berat barang yang ada di gudang. Setiap kali terjadi perubahan data (penambahan, pengurangan, atau pengubahan), sistem secara otomatis menyimpan kondisi tumpukan ke dalam file teks gudang.txt sehingga data tetap tersedia ketika program dijalankan kembali pada sesi berikutnya.

Sistem juga menerapkan validasi kapasitas: total berat seluruh barang dalam tumpukan tidak diperbolehkan mencapai atau melebihi 100 kg. Validasi ini diterapkan baik saat menambah barang baru maupun saat mengubah berat barang yang sudah ada, sehingga konsistensi batas maksimum gudang tetap terjaga dalam kedua operasi tersebut.

2.2 Struktur Data yang Digunakan

Struktur Data	Fungsi
Linked List (Stack)	Menyimpan seluruh data barang gudang secara LIFO; setiap node (Barang) menyimpan nama, berat, indeks masuk, dan pointer ke node berikutnya.
File txt	Menyimpan data barang secara permanen (persistence) agar tidak hilang ketika program ditutup, termasuk menyimpan nilai counter indeks terakhir.
List Python	Digunakan sebagai struktur perantara untuk menyalin data dari linked list agar dapat diurutkan (sorting) menggunakan fungsi bawaan Python, karena linked list tidak dapat di-sort secara langsung.

Alasan Pemilihan Struktur Data

Stack dipilih sebagai struktur data utama karena merepresentasikan kondisi fisik nyata dari penumpukan barang di gudang. Barang yang paling terakhir dimasukkan akan selalu berada pada posisi paling atas, dan barang tersebut pula yang harus dikeluarkan lebih dahulu sebelum barang di bawahnya dapat diakses. Prinsip Last In First Out (LIFO) ini sejalan dengan cara kerja operasi push (menambah barang ke atas tumpukan) dan pop (mengambil barang dari atas tumpukan) yang diimplementasikan dalam program.

Implementasi Stack menggunakan Linked List (bukan array/list Python biasa) dipilih karena ukuran tumpukan bersifat dinamis — jumlah barang yang masuk tidak diketahui sejak awal dan dapat bertambah maupun berkurang sewaktu-waktu. Dengan Linked List, penambahan dan penghapusan node dapat dilakukan dalam kompleksitas waktu $O(1)$ tanpa perlu menggeser posisi elemen lain, berbeda dengan array statis yang memerlukan alokasi ukuran tetap di awal.

2.3 Algoritma yang Digunakan

Searching

Sequential Search (Linear Search) digunakan pada dua fitur:

- Fitur pencarian barang (cari) — menyalisir seluruh node dari atas ke bawah tanpa berhenti di tengah jalan, karena tujuannya adalah mengumpulkan semua barang yang namanya mengandung kata kunci pencarian (partial match, case-insensitive).
- Fitur ubah informasi barang (ubah_informasi) — menyalisir node satu per satu berdasarkan nilai indeks unik, dan berhenti (early exit) begitu barang dengan indeks yang dicari ditemukan.

Alasan penggunaan: Data tersimpan dalam bentuk Linked List yang hanya dapat diakses secara sekuensial mulai dari node teratas (top), berbeda dengan array yang mendukung pengaksesan acak (random access) berdasarkan posisi indeks. Karena keterbatasan ini, algoritma pencarian biner (Binary Search) yang membutuhkan akses acak dan data terurut tidak dapat diterapkan langsung pada struktur Linked List tanpa terlebih dahulu menyalinnya ke struktur array. Sequential Search dengan kompleksitas waktu $O(n)$ merupakan pilihan yang paling sesuai dan paling sederhana untuk diimplementasikan pada struktur Stack berbasis Linked List.

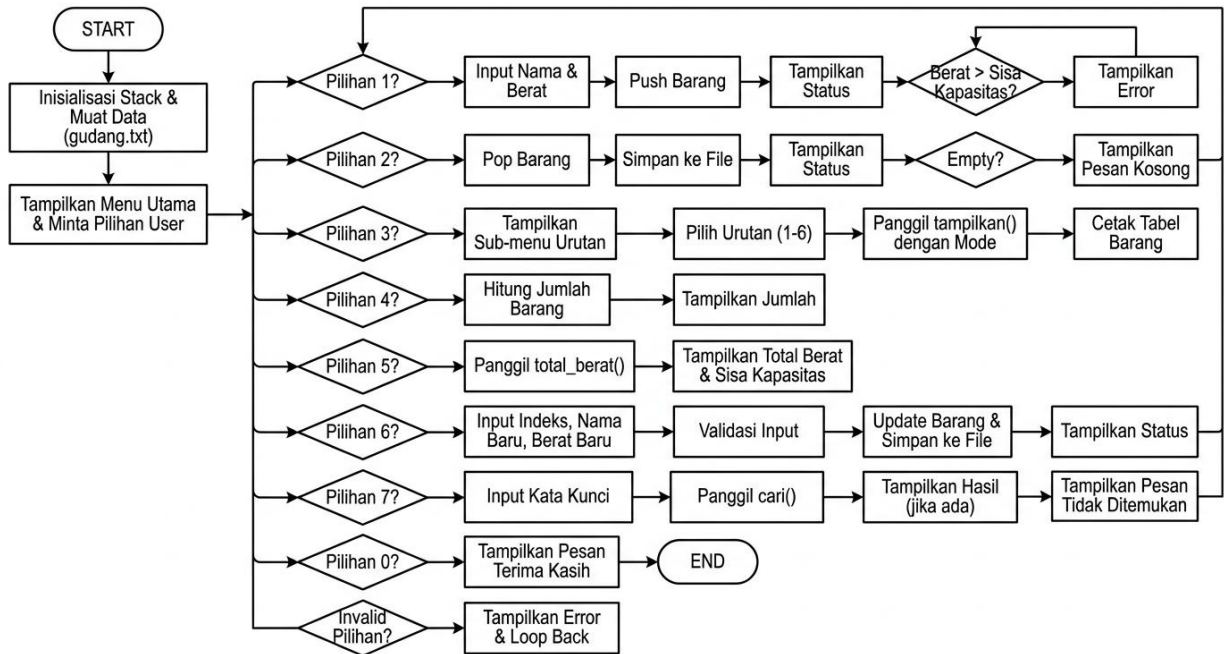
Sorting

Pengurutan data barang menggunakan fungsi bawaan Python `list.sort()`, yang secara internal menerapkan algoritma Timsort — algoritma hybrid yang menggabungkan Merge Sort dan Insertion Sort, dengan kompleksitas waktu rata-rata maupun terburuk $O(n \log n)$.

Alasan penggunaan: Karena Linked List tidak dapat diurutkan secara langsung menggunakan fungsi sort bawaan Python, seluruh node terlebih dahulu disalin ke dalam list Python sementara melalui fungsi pembantu (`_ke_list`). Setelah itu, list tersebut diurutkan berdasarkan salah satu dari tiga kriteria: berat barang, indeks urutan masuk, atau nama barang secara alfabet, dengan opsi ascending maupun descending pada masing-masing kriteria. Pendekatan ini dipilih karena lebih efisien dan tidak memerlukan implementasi algoritma sorting secara manual, sementara tetap memenuhi kebutuhan fungsional aplikasi untuk menampilkan

data dari berbagai sudut pandang tanpa mengubah urutan fisik tumpukan yang sesungguhnya.

2.4 Diagram Alir Program (Flowchart)



2.5 Struktur Program

Program disusun dalam satu file utama dengan struktur sebagai berikut:

- ManajemenGudang.py — berisi seluruh logika program:
- Class Barang — representasi satu node/elemen barang dalam Stack (nama, berat, indeks, pointer next)
- Class Stack — struktur data utama beserta seluruh method CRUD, searching, sorting, dan file handling
- Fungsi main() — antarmuka menu CLI yang menangani input pengguna dan memanggil method Stack yang sesuai
- gudang.txt — file penyimpanan data barang yang dibuat dan diperbarui otomatis oleh program

BAB III. IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama

Saat program dijalankan, pengguna disuguhkan menu utama berisi delapan pilihan: Masukkan Barang, Keluarkan Barang, Lihat Semua Barang, Jumlah Barang, Total Berat Barang, Ubah Informasi Barang, Cari Barang, dan Keluar. Program akan terus menampilkan menu ini dalam sebuah perulangan (loop) hingga pengguna memilih untuk keluar.

```
[INFO] Data berhasil dimuat dari 'gudang.txt'.
      6 barang | 49.0 kg

=====
  MANAJEMEN TUMPUKAN GUDANG BARANG
=====

1. Masukkan Barang
2. Keluarkan Barang
3. Lihat Semua Barang
4. Jumlah Barang
5. Total Berat Barang
6. Ubah Informasi Barang
7. Cari Barang
0. Keluar

Pilih menu (0-7):
```

3.2 Fitur Create

Fitur ini diakses melalui menu 1. Pengguna diminta memasukkan nama dan berat barang. Sebelum barang ditambahkan, program memeriksa apakah penambahan tersebut akan membuat total berat tumpukan menyentuh atau melebihi kapasitas maksimum (100 kg). Jika kapasitas terlampaui, barang ditolak beserta informasi sisa kapasitas yang tersedia. Jika valid, barang baru ditambahkan pada posisi teratas tumpukan (top) dan diberi nomor indeks unik berdasarkan counter yang terus bertambah.

```
Pilih menu (0-7): 1
Masukkan nama barang: karpet
Masukkan berat barang (kg): 2

Barang 'karpet' berhasil ditambahkan!
Total berat sekarang: 51.0 kg / 99 kg
Data tersimpan ke 'gudang.txt'.
```

3.3 Fitur Read

Fitur ini diakses melalui menu 3 dan dijelaskan lebih lanjut pada bagian 3.7 karena terintegrasi langsung dengan fitur sorting.

```
Pilih menu (0-7): 3

--- Urutan Tampilan ---
1. Berat Barang - Ringan → Berat
2. Berat Barang - Berat → Ringan
3. Indeks Masuk - Ascending
4. Indeks Masuk - Descending
5. Alphabet - A → Z
6. Alphabet - Z → A
Pilih urutan (1-6): 5

===== DAFTAR BARANG: URUTAN ALPHABET - A → Z =====
-----
No   Indeks  Nama Barang      Berat
-----
1    29      batu             8.0 kg
2    21      batu             7.0 kg
3    31      besi             4.0 kg
4    33      emas             24.0 kg
5    34      karpet           2.0 kg
6    30      keramik          4.0 kg
7    22      marbel           2.0 kg
-----
Total berat: 51.0 kg / 99 kg
```

3.4 Fitur Update

Fitur ini diakses melalui menu 6. Pengguna memasukkan indeks barang yang ingin diubah beserta nama dan berat baru. Program mencari barang

dengan indeks yang sesuai menggunakan sequential search, lalu memperbarui nama dan beratnya. Sebelum perubahan disimpan, program memvalidasi apakah berat baru akan membuat total tumpukan melebihi kapasitas maksimum; jika demikian, perubahan ditolak.

```
Pilih menu (0-7): 6
Masukkan indeks barang yang ingin diubah: 22
Masukkan nama baru: marbel
Masukkan berat baru (kg): 23

Barang ditemukan: marbel (Berat: 2.0 kg)
Informasi barang berhasil diubah.
```

3.5 Fitur Delete

Fitur ini diakses melalui menu 2. Karena struktur datanya adalah Stack, barang yang dikeluarkan selalu barang yang berada pada posisi teratas tumpukan (pop), sesuai prinsip LIFO. Jika tumpukan kosong, program menampilkan pesan bahwa tidak ada barang yang dapat dikeluarkan.

```
Pilih menu (0-7): 2

Barang 'karpet' berhasil dihapus.
Berat barang: 2.0 kg
Data tersimpan ke 'gudang.txt'.
```

3.6 Fitur Searching

Fitur ini diakses melalui menu 7. Algoritma yang digunakan adalah Sequential Search dengan pencocokan sebagian (partial match) dan tidak peka huruf besar/kecil (case-insensitive). Program menyisir seluruh barang dalam tumpukan dan mengumpulkan setiap barang yang nama-nya mengandung kata kunci yang dimasukkan pengguna, sehingga pengguna tidak perlu mengetik nama barang secara lengkap dan tepat untuk menemukannya.

Contoh: jika pengguna mencari kata kunci “kar”, maka barang dengan nama “Kardus Kecil”, “Kardus Besar”, maupun “Kayu Kardus” akan sama-sama ditemukan, karena ketiganya mengandung substring “kar” pada namanya.

```
Pilih menu (0-7): 7
Masukkan kata kunci pencarian: b

===== HASIL PENCARIAN: 'b' =====
Ditemukan 4 barang:

-----
No   Indeks  Nama Barang      Berat
-----
1    31      besi             4.0 kg
2    29      batu             8.0 kg
3    22      marbel           23.0 kg
4    21      batu             7.0 kg
-----
Total berat: 70.0 kg / 99 kg
```

3.7 Fitur Sorting

Fitur ini diakses melalui menu 3 dan menyediakan enam pilihan urutan tampilan: berat barang (ringan ke berat / berat ke ringan), indeks urutan masuk (ascending / descending), dan nama barang secara alfabet (A ke Z / Z ke A). Algoritma yang digunakan adalah fungsi `sort()` bawaan Python (Timsort) yang diterapkan pada salinan data dalam bentuk list, sehingga urutan fisik tumpukan yang sesungguhnya tidak berubah — fitur ini hanya mengubah cara data ditampilkan kepada pengguna.

```
Pilih menu (0-7): 3

--- Urutan Tampilan ---
1. Berat Barang - Ringan → Berat
2. Berat Barang - Berat → Ringan
3. Indeks Masuk - Ascending
4. Indeks Masuk - Descending
5. Alphabet - A → Z
6. Alphabet - Z → A
Pilih urutan (1-6): 5

===== DAFTAR BARANG: URUTAN ALPHABET - A → Z =====
-----
No   Indeks  Nama Barang      Berat
-----
1    29      batu             8.0 kg
2    21      batu             7.0 kg
3    31      besi             4.0 kg
4    33      emas             24.0 kg
5    34      karpet           2.0 kg
6    30      keramik          4.0 kg
7    22      marbel           2.0 kg
-----
Total berat: 51.0 kg / 99 kg
```

3.8 Penyimpanan Data (File Handling)

Data barang disimpan dalam file bernama gudang.txt menggunakan format teks dengan pemisah karakter “|” (pipe) pada setiap baris, dengan urutan: indeks|nama|berat. Baris pertama file secara khusus menyimpan nilai counter indeks terakhir dengan format #counter:<nilai>, yang digunakan untuk memastikan nomor indeks barang baru tetap berlanjut meskipun program ditutup dan dijalankan kembali.

Cara penyimpanan: setiap kali terjadi perubahan data (tambah, hapus, atau ubah), seluruh node dalam tumpukan dikumpulkan ke dalam list sementara, kemudian dituliskan ke file dari bawah ke atas (reversed) agar urutan tumpukan tetap konsisten saat dimuat ulang.

Cara pembacaan: saat program dijalankan, file dibaca baris demi baris. Baris yang diawali #counter: digunakan untuk memulihkan nilai counter,

sedangkan baris lainnya dipecah menggunakan `split("|")` menjadi tiga bagian dan langsung dimasukkan kembali ke dalam tumpukan.

Isi file `gudang.txt`:

```
#counter:33
```

```
21|batu|7.0
```

```
22|marbel|2.0
```

```
29|batu|8.0
```

```
30|keramik|4.0
```

```
31|besi|4.0
```

```
33|emas|24.0
```

3.9 Validasi Input dan Error Handling

Kesalahan Input	Penanganan
Huruf dimasukkan pada input berat (seharusnya angka)	Program menangkap <code>ValueError</code> dan meminta pengguna memasukkan ulang nilai berat.
Berat bernilai nol atau negatif	Program menolak nilai tersebut dan menampilkan pesan agar memasukkan berat lebih dari 0 kg.
Total berat akan menyentuh atau melebihi 100 kg (saat tambah maupun ubah barang)	Program menolak operasi dan menampilkan informasi berat saat ini serta sisa kapasitas yang tersedia.
Barang/indeks tidak ditemukan saat ubah informasi	Program menampilkan pesan bahwa barang dengan indeks tersebut tidak ditemukan.
Kata kunci pencarian tidak cocok dengan nama barang manapun	Program menampilkan pesan bahwa tidak ada barang yang cocok dengan kata kunci tersebut.
File <code>gudang.txt</code> belum tersedia	Program menangkap <code>FileNotFoundError</code> dan memulai tumpukan dalam kondisi kosong tanpa menghentikan eksekusi.
Pilihan menu di luar rentang yang tersedia	Program menampilkan pesan pilihan tidak valid dan menampilkan ulang menu utama.

BAB IV. PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

No	Skenario	Hasil
1	Menambah barang dengan berat valid (total < 100 kg)	Berhasil

2	Menambah barang yang membuat total ≥ 100 kg	Berhasil
3	Mengeluarkan barang (pop) saat tumpukan kosong	Berhasil
4	Mengeluarkan barang (pop) saat tumpukan berisi	Berhasil
5	Mengubah informasi barang dengan indeks valid	Berhasil
6	Mengubah berat barang hingga melebihi 100 kg	Berhasil
7	Mencari barang dengan kata kunci sebagian nama	Berhasil
8	Mencari barang dengan kata kunci yang tidak ada	Berhasil
9	Menampilkan barang terurut berdasarkan berat, indeks, dan alfabet (ascending & descending)	Berhasil
10	Menutup dan membuka kembali program	Berhasil
11	Memasukkan huruf pada input berat barang	Berhasil

4.2 Kelebihan Sistem

1. Data tersimpan secara permanen menggunakan file TXT, sehingga tidak hilang ketika program ditutup, termasuk nomor indeks unik tiap barang.
2. Validasi kapasitas maksimum diterapkan secara konsisten pada operasi tambah maupun ubah informasi barang, mencegah kelebihan muatan yang dapat membahayakan kestabilan tumpukan.
3. Fitur pencarian mendukung pencocokan sebagian nama (partial match) dan tidak peka huruf besar/kecil, sehingga lebih fleksibel digunakan meski pengguna tidak mengetahui nama barang secara lengkap dan tepat.
4. Fitur sorting menyediakan enam mode urutan tampilan tanpa mengubah urutan fisik tumpukan, memberi pengguna fleksibilitas dalam membaca data dari berbagai sudut pandang.
5. Penanganan kesalahan input (validasi angka, nilai negatif, file belum tersedia) membuat program tidak mudah mengalami crash saat digunakan.

4.4 Keterbatasan Sistem

1. Masih berbasis Command Line Interface (CLI), belum memiliki antarmuka grafis (GUI) yang lebih ramah bagi pengguna umum.

2. Belum mendukung sistem basis data (database) sehingga skalabilitas pengelolaan data dalam jumlah sangat besar masih terbatas dibandingkan solusi berbasis database.
3. Belum mendukung multi pengguna (multi-user) atau akses bersamaan (concurrent access) terhadap file gudang.txt, sehingga rawan terjadi konflik data jika diakses lebih dari satu proses secara bersamaan.
4. Pencarian dan pengurutan menggunakan kompleksitas waktu $O(n)$ dan $O(n \log n)$ yang cukup efisien untuk skala kecil-menengah, namun dapat menjadi kurang optimal jika jumlah barang bertambah sangat besar dibanding struktur data yang mendukung pengaksesan acak.

BAB V. KENDALA DAN SOLUSI

Kendala	Solusi
Validasi kapasitas maksimum sempat tidak berfungsi karena fungsi penghitung total berat bercampur dengan fungsi penampil (print), sehingga nilai yang dikembalikan selalu kosong (None).	Memisahkan fungsi penghitung total berat menjadi dua: satu fungsi internal yang hanya mengembalikan nilai angka, dan satu fungsi lain yang khusus menampilkan informasi ke pengguna.
Data hilang setiap kali program ditutup karena awalnya tidak ada mekanisme penyimpanan permanen.	Menambahkan mekanisme file handling menggunakan file TXT yang otomatis memuat data saat program dibuka dan menyimpan data setiap kali terjadi perubahan.
Pengurutan (sorting) tidak dapat langsung diterapkan pada struktur Linked List.	Menyalin data dari Linked List ke list Python sementara sebelum diurutkan, lalu menampilkannya tanpa mengubah struktur tumpukan asli.
Pengubahan informasi barang berdasarkan nama berisiko ambigu apabila terdapat dua barang dengan nama yang sama.	Menambahkan atribut indeks unik pada setiap barang berdasarkan urutan masuk, sehingga perubahan informasi dilakukan berdasarkan indeks yang dipastikan tidak ada duplikatnya.
Input pengguna yang tidak valid (huruf pada kolom angka) berpotensi menyebabkan program berhenti tiba-tiba (crash).	Menambahkan blok try-except pada seluruh input numerik untuk menangkap ValueError dan meminta pengguna memasukkan ulang.

Pengalaman menantang yang dihadapi selama pengerjaan proyek ini, antara lain, terletak pada perancangan logika penyimpanan file agar urutan tumpukan (yang bersifat LIFO) tetap konsisten setiap kali data dimuat ulang dari file, mengingat urutan penulisan ke file dan urutan pembacaan kembali harus saling berkebalikan agar posisi top tidak tertukar.

BAB VI. PEMBAGIAN TUGAS ANGGOTA

Nama	Tugas
Muhammad Zeya Rieqi	CRUD dan File Handling — implementasi class Barang, Stack, fungsi push, pop, ubah_informasi, serta mekanisme penyimpanan dan pemuatan data dari file gudang.txt
Nelson Arsel Warae	Searching dan Sorting — implementasi fungsi cari (sequential search) dan tampilkan (sorting berdasarkan berat, indeks, dan alfabet)
Muhammad Danish Rahmanu Ghaisan	Struktur Program dan Pengujian — perancangan menu utama (main), pengujian seluruh skenario pada Bab IV, serta penyusunan laporan

BAB VII. KESIMPULAN DAN SARAN

Kesimpulan

- Struktur data Stack berbasis Linked List berhasil diimplementasikan untuk merepresentasikan pengelolaan tumpukan barang gudang secara realistis sesuai prinsip Last In First Out (LIFO).
- Fitur penyimpanan data permanen (file handling), validasi kapasitas, searching, dan sorting berhasil diintegrasikan ke dalam satu aplikasi CLI yang dapat digunakan untuk mengelola data barang gudang secara lebih terstruktur dibandingkan pencatatan manual.
- Pengujian terhadap seluruh skenario pada Bab IV menunjukkan bahwa program berjalan sesuai dengan rancangan, termasuk pada kondisi-kondisi tepi (edge case) seperti tumpukan kosong, kapasitas penuh, dan input tidak valid.

Saran Pengembangan

1. Mengembangkan antarmuka grafis (GUI) atau antarmuka berbasis web agar program lebih mudah digunakan oleh pengguna non-teknis.
 2. Mengintegrasikan sistem basis data (misalnya SQLite) sebagai pengganti file TXT untuk meningkatkan skalabilitas dan keandalan penyimpanan data dalam jumlah besar.
 3. Menambahkan fitur multi-gudang (multiple stacks) untuk mengelola lebih dari satu lokasi penyimpanan secara bersamaan.
 4. Menambahkan fitur riwayat (log) setiap transaksi tambah dan keluar barang untuk keperluan audit dan pelacakan.
-

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.
 - [2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Data Structures and Algorithms in Python. Hoboken, NJ: Wiley, 2013.
 - [3] B. Miller and D. Ranum, Problem Solving with Algorithms and Data Structures Using Python. Franklin, Beedle & Associates, 2011.
 - [4] Python Software Foundation, “Python 3 Documentation — Sorting HOW TO,” Python.org. [Online]. Available: <https://docs.python.org/3/howto/sorting.html>
-

LAMPIRAN

Lampiran A. Link GitHub

https://github.com/Cain-BR/ASD_J0403251082_MuhammadZeyaRieqi/tree/main/ASD_B1_5

Lampiran B. Screenshot Program

```
[INFO] Data berhasil dimuat dari 'gudang.txt'.
      6 barang | 49.0 kg

=====
MANAJEMEN TUMPUKAN GUDANG BARANG
=====
1. Masukkan Barang
2. Keluarkan Barang
3. Lihat Semua Barang
4. Jumlah Barang
5. Total Berat Barang
6. Ubah Informasi Barang
7. Cari Barang
0. Keluar

Pilih menu (0-7):
```

Lampiran C. Struktur Folder Proyek

Berkas/Folder	Keterangan
ManajemenGudang.py	Berkas utama program (class Barang, class Stack, fungsi main)
Gudang.txt	Berkas penyimpanan data barang (dibuat otomatis oleh program)

Lampiran D. Commit History GitHub

Commit 1e2244f

Cain-BR committed on Feb 18

Projek1

main 1 parent 1877728 commit 1e2244f

1 file changed

Filter files... Projek_kel-5 MaTuBa.py

Search within code

Projek_kel-5/MaTuBa.py

Whitespace-only changes.

Comments 0

Please sign in to comment.

Commit 475ff9d

Cain-BR committed on Feb 18

Projek2

main 1 parent 1e2244f commit 475ff9d

1 file changed +16

Filter files... Projek_kel-5 MaTuBa.py

Search within code

Projek_kel-5/MaTuBa.py +16

```
@@ -0,0 +1,16 @@
1 + #Projek kelompok 5
2 + #Manajemen Tumpukan Gudang Barang Berat Menggunakan Struktur Data Stack
3 + #Muhammad Danish Rahmamu Ghaisan (J0403251124)
4 + #Muhammad Zeya Rieqi (J0403251082)
5 + #Nelson Arsel Warae (J0403251061)
6 + #Algoritma dan Struktur Data
7 + #Kategori: Pelajaran Pengantar Luruh
```

Commit 3400cbe

Cain-BR committed on Mar 4

Projek3

main 1 parent 3c5439e commit 3400cbe

1 file changed +59 -1

Filter files... Projek_kel-5 MaTuBa.py

Search within code

Projek_kel-5/MaTuBa.py +59 -1

```
@@ -13,4 +13,62 @@
13 13 class barang:
14 14     def __init__(self, nama, berat):
15 15         self.nama = nama
16 -         self.berat = berat
16 +         self.berat = berat
```

Commit **c3c2fffd**

Cain-BR

committed on Apr 15

Projek4

main

1 parent [a7607fa](#) commit [c3c2fffd](#)

1 file changed

Filter files...

Projek_kel-5

ManajemenGudang.py

Search within code

Projek_kel-5/MaTuBa.py → Projek_kel-5/ManajemenGudang.py

File renamed without changes.

Comments 0

Lock conversation

Comment

Commit **3a41d30**

Cain-BR

committed on May 13

Projek5

main

1 parent [484d527](#) commit [3a41d30](#)

1 file changed

Filter files...

Projek_kel-5

ManajemenGudang.py

Search within code

Projek_kel-5/ManajemenGudang.py

+203 -32

@@ -1,74 +1,245 @@

1 - #Projek kelompok 5

2 - #Manajemen Tumpukan Gudang Barang Berat Menggunakan Struktur Data Stack

3 - #Muhammad Danish Rahmahu Ghaisan (J0403251124)

4 - #Muhammad Zeya Rieqi (J0403251082)

5 - #Nelson Arsel Warae (J0403251061)

6 - #Algoritma dan Struktur Data

Commit **9c5486a**

Cain-BR

committed on May 13

Proyek6

main

1 parent [9fe0716](#) commit [9c5486a](#)

2 files changed

Filter files...

Projek_kel-5

ManajemenGudang.py

gudang.txt

Search within code

Projek_kel-5/ManajemenGudang.py

+143 -63

@@ -27,9 +27,13 @@ def __init__(self, nama, berat):

27 27 # CLASS STACK

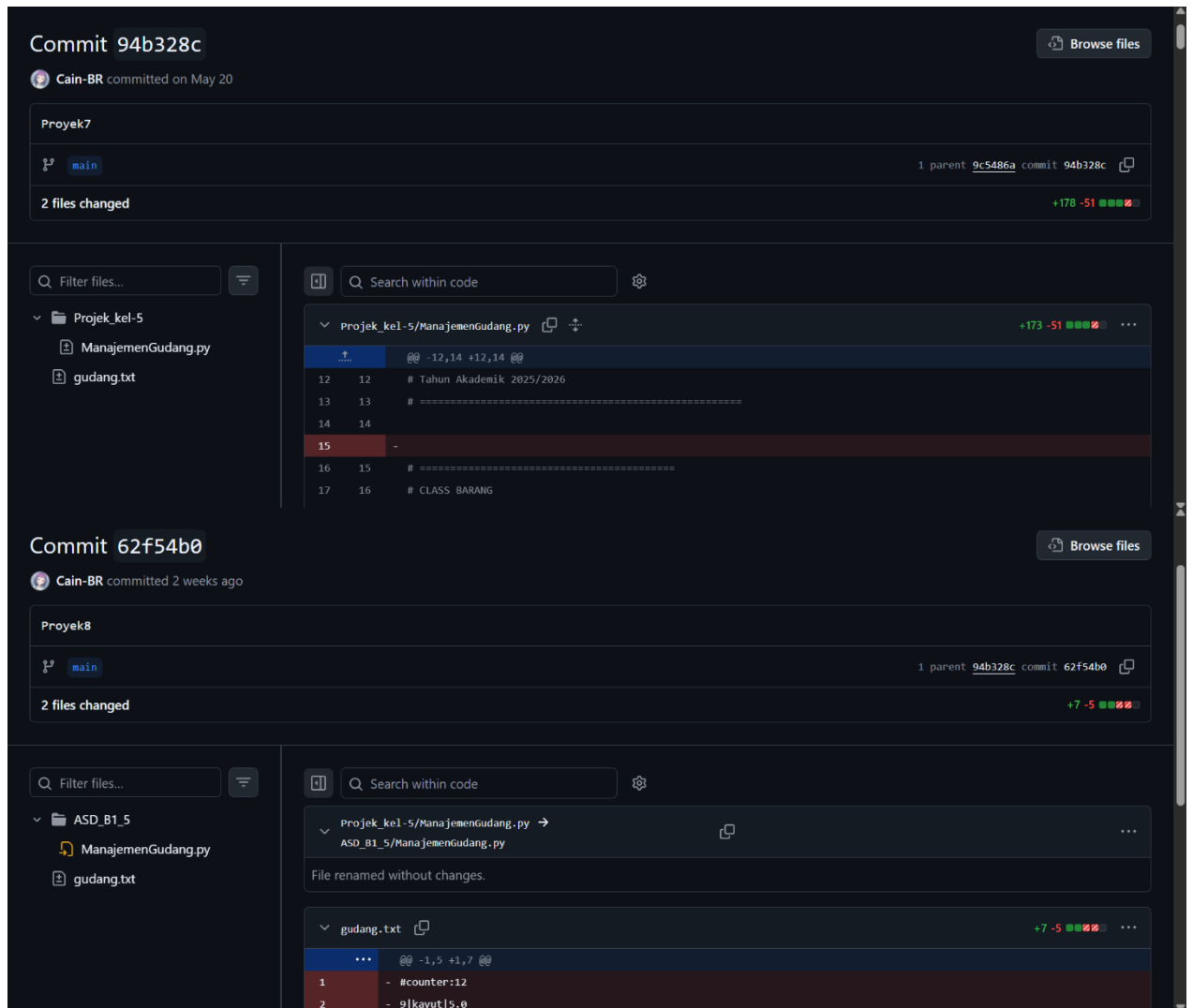
28 28 # =====

29 29 class Stack:

30 - def __init__(self, max_size=5):

30 + def __init__(self, max_size=100, nama_file="gudang.txt"):

31 31 self.top = None



Lampiran E. Source Code Utama

```
#  
=====
```

==

```
# Proyek Kelompok 5  
# Manajemen Tumpukan Gudang Barang Berat  
# Menggunakan Struktur Data Stack  
#  
# Muhammad Danish Rahmanu Ghaisan (J0403251124)  
# Muhammad Zeya Rieqi (J0403251082)  
# Nelson Arsel Warae (J0403251061)  
#  
# Mata Kuliah : Algoritma dan Struktur Data  
# Program Studi : Teknologi Rekayasa Perangkat Lunak
```



```

# Tahun Akademik 2025/2026
#
=====

# =====
# CLASS BARANG
# =====

class Barang:
    def __init__(self, nama, berat, indeks):
        self.nama = nama
        self.berat = berat
        self.indeks = indeks # urutan masuk: barang pertama = 1
        self.next = None

# =====
# CLASS STACK
# =====

class Stack:
    def __init__(self, max_size=100, nama_file="gudang.txt"):
        self.top = None
        self.max_size = max_size
        self.nama_file = nama_file
        self.indeks_counter = 0 # cacah total barang yang pernah masuk

        # Muat data dari file saat program dimulai
        self._muat_dari_file()

    # Mengecek apakah stack kosong
    def is_empty(self):
        return self.top is None

    # Menghitung jumlah barang
    def jumlah_barang(self):
        count = 0
        current = self.top
        while current:
            count += 1
            current = current.next
        return count

# =====
# HITUNG TOTAL BERAT (internal, return nilai)

```

```

# =====
def _hitung_total_berat(self):
    total = 0
    current = self.top
    while current:
        total += current.berat
        current = current.next
    return total

# =====
# SIMPAN KE FILE TXT
# Format tiap baris: nama|berat
# Urutan disimpan dari bawah ke atas
# agar saat dimuat ulang, top tetap benar
# =====
def _simpan_ke_file(self):
    try:
        barang_list = []
        current = self.top
        while current:
            barang_list.append(current)
            current = current.next

        # Format: indeks|nama|berat
        # Tulis dari bawah ke atas agar urutan stack tetap benar saat dimuat
        with open(self.nama_file, "w") as f:
            f.write(f"#counter: {self.indeks_counter}\n")
            for barang in reversed(barang_list):
                f.write(f"{barang.indeks}|{barang.nama}|{barang.berat}\n")

    except IOError as e:
        print(f"\n[ERROR] Gagal menyimpan ke file: {e}")

# =====
# MUAT DARI FILE TXT
# =====
def _muat_dari_file(self):
    try:
        with open(self.nama_file, "r") as f:
            lines = f.readlines()

        for line in lines:
            line = line.strip()

```

```

        if not line:
            continue

        # Baca counter dari baris pertama
        if line.startswith("#counter:"):
            self.indeks_counter = int(line.split(":")[1])
            continue

        parts = line.split("|")
        if len(parts) == 3:
            indeks = int(parts[0])
            nama = parts[1]
            berat = float(parts[2])
            barang_baru = Barang(nama, berat, indeks)
            barang_baru.next = self.top
            self.top = barang_baru

        if not self.is_empty():
            print(f'[INFO] Data berhasil dimuat dari '{self.nama_file}'.')
            print(f'                {self.jumlah_barang()} barang |
{self._hitung_total_berat()} kg")

    except FileNotFoundError:
        pass
    except (IOError, ValueError) as e:
        print(f'\n[ERROR] Gagal membaca file: {e}')

# =====
# PUSH -> Menambahkan barang
# =====
def push(self, nama, berat):

    # Cek kapasitas
    berat_sekarang = self._hitung_total_berat()
    if berat_sekarang + berat >= self.max_size:
        sisa = self.max_size - berat_sekarang
        print(f'\nMuatan terlalu berat! Barang tidak bisa ditambahkan.')
        print(f'Berat saat ini : {berat_sekarang} kg")
        print(f'Sisa kapasitas : {sisa} kg")
        print(f'Batas maksimum : {self.max_size - 1} kg")
        return

    # Buat node baru dengan indeks berikutnya

```

```

        self.indeks_counter += 1
        barang_baru = Barang(nama, berat, self.indeks_counter)
        barang_baru.next = self.top
        self.top = barang_baru

    # Simpan ke file
    self._simpan_ke_file()

    print(f"\nBarang '{nama}' berhasil ditambahkan!")
    print(f"Total berat sekarang: {self._hitung_total_berat()} kg /
{self.max_size - 1} kg")
    print(f"Data tersimpan ke '{self.nama_file}'.")

# =====
# POP -> Menghapus barang teratas
# =====
def pop(self):

    if self.is_empty():
        print("Stack kosong! Tidak ada barang.")
        return

    nama = self.top.nama
    berat = self.top.berat
    self.top = self.top.next

    # Simpan ke file
    self._simpan_ke_file()

    print(f"\nBarang '{nama}' berhasil dihapus.")
    print(f"Berat barang: {berat} kg")
    print(f"Data tersimpan ke '{self.nama_file}'.")

# =====
# HELPER: ambil semua barang sebagai list
# =====
def _ke_list(self):
    hasil = []
    current = self.top
    while current:
        hasil.append(current)
        current = current.next
    return hasil

```

```

# =====
# HELPER: cetak tabel barang dari list
# =====
def _cetak_tabel(self, barang_list):
    print("-" * 50)
    print(f'{"No":<4} {"Indeks":<8} {"Nama Barang":<20} {"Berat":>8}')
    print("-" * 50)
    for no, b in enumerate(barang_list, 1):
        print(f'{"no":<4} {"b.indeks":<8} {"b.nama":<20} {"b.berat":>6} kg')
    print("-" * 50)
    print(f'Total berat: {self._hitung_total_berat()} kg / {self.max_size - 1} kg')

# =====
# MENAMPILKAN SEMUA BARANG
# =====
def tampilkan(self, mode="tumpukan", ascending=True):
    if self.is_empty():
        print("\nTumpukan barang kosong.")
        return

    barang_list = self._ke_list()

    if mode == "tumpukan":
        barang_list.sort(key=lambda b: b.berat, reverse=not ascending)
        arah = "Ringan → Berat" if ascending else "Berat → Ringan"
        label = f"URUTAN BERAT BARANG — {arah}"

    elif mode == "indeks":
        barang_list.sort(key=lambda b: b.indeks, reverse=not ascending)
        arah = "Ascending" if ascending else "Descending"
        label = f"URUTAN INDEKS MASUK — {arah}"

    elif mode == "alphabet":
        barang_list.sort(key=lambda b: b.nama.lower(), reverse=not ascending)
        arah = "A → Z" if ascending else "Z → A"
        label = f"URUTAN ALPHABET — {arah}"

    else:
        print("Mode tampil tidak dikenal.")
        return

```

```

        print(f'\n===== DAFTAR BARANG: {label} =====')
        self._cetak_tabel(barang_list)

# =====
# CARI BARANG (partial match, case-insensitive)
# =====
def cari(self, kata_kunci):
    if self.is_empty():
        print("\nStack kosong! Tidak ada barang.")
        return

    kata_kunci = kata_kunci.strip().lower()

    if not kata_kunci:
        print("\nKata kunci tidak boleh kosong.")
        return

    # Kumpulkan semua barang yang cocok
    hasil = []
    current = self.top
    while current:
        if kata_kunci in current.nama.lower():
            hasil.append(current)
            current = current.next

    if not hasil:
        print(f'\nTidak ada barang yang cocok dengan '{kata_kunci}'.')
        return

    print(f'\n===== HASIL PENCARIAN: '{kata_kunci}' =====')
    print(f'Ditemukan {len(hasil)} barang:\n")
    self._cetak_tabel(hasil)

# =====
# TOTAL BERAT BARANG
# =====
def total_berat(self):
    if self.is_empty():
        print("Tidak ada barang dalam stack.")
        return

```

```

        total = self._hitung_total_berat()
        print(f"\nTotal berat seluruh barang : {total} kg")
        print(f"Kapasitas maksimum      : {self.max_size - 1} kg")
        print(f"Sisa kapasitas          : {self.max_size - 1 - total} kg")

# =====
# UBAH INFORMASI BARANG
# =====
def ubah_informasi(self, ubah_indeks, nama_baru, berat_baru):

    if self.is_empty():
        print("Stack kosong! Tidak ada barang.")
        return

    try:
        ubah_indeks = int(ubah_indeks)
    except ValueError:
        print("Indeks harus berupa angka!")
        return

    current = self.top
    while current:
        if current.indeks == ubah_indeks:
            print(f"\nBarang ditemukan: {current.nama} (Berat: {current.berat} kg)")
            current.nama = nama_baru
            current.berat = berat_baru
            self._simpan_ke_file()
            print("Informasi barang berhasil diubah.")
            return
        current = current.next

    print(f"\nBarang dengan indeks {ubah_indeks} tidak ditemukan.")

# =====
# PROGRAM UTAMA
# =====
def main():

    stack = Stack()

    while True:

```

```
print("\n" + "=" * 45)
print(" MANAJEMEN TUMPUKAN GUDANG BARANG ")
print("=" * 45)
```

```
print("1. Masukkan Barang")
print("2. Keluarkan Barang")
print("3. Lihat Semua Barang")
print("4. Jumlah Barang")
print("5. Total Berat Barang")
print("6. Ubah Informasi Barang")
print("7. Cari Barang")
print("0. Keluar")
```

```
pilihan = input("\nPilih menu (0-7): ")
```

```
# =====
# MENU 1 -> TAMBAH BARANG
# =====
```

```
if pilihan == "1":
    nama = input("Masukkan nama barang: ")
    try:
        berat = float(input("Masukkan berat barang (kg): "))
        if berat <= 0:
            print("Masukkan berat yang valid (lebih dari 0 kg)!")
        else:
            stack.push(nama, berat)
    except ValueError:
        print("Input berat harus berupa angka!")
```

Saat user memilih menu 1, setelah memasukkan nama dan berat, program akan mencoba menambahkan barang ke stack. Jika berat yang dimasukkan melebihi kapasitas maksimum, program akan memberikan informasi tentang berat saat ini, sisa kapasitas, dan batas maksimum. Jika berhasil ditambahkan, program akan menampilkan total berat sekarang dan konfirmasi bahwa data telah disimpan ke file.

```
# =====
# MENU 2 -> HAPUS BARANG
# =====
```

```
elif pilihan == "2":
    stack.pop()
```


Saat user memilih menu 2, program akan mencoba menghapus barang teratas dari stack. Jika stack kosong, program akan memberitahu bahwa tidak ada barang yang bisa dihapus. Jika berhasil dihapus, program akan menampilkan nama dan berat barang yang dihapus, serta konfirmasi bahwa data telah disimpan ke file.

```
# =====
# MENU 3 -> TAMPILKAN BARANG
# =====
elif pilihan == "3":

    print("\n--- Urutan Tampilan ---")
    print("1. Berat Barang - Ringan → Berat")
    print("2. Berat Barang - Berat → Ringan")
    print("3. Indeks Masuk - Ascending")
    print("4. Indeks Masuk - Descending")
    print("5. Alphabet - A → Z")
    print("6. Alphabet - Z → A")

    sub = input("Pilih urutan (1-6): ")

    if sub == "1":
        stack.tampilkan(mode="tumpukan", ascending=True)
    elif sub == "2":
        stack.tampilkan(mode="tumpukan", ascending=False)
    elif sub == "3":
        stack.tampilkan(mode="indeks", ascending=True)
    elif sub == "4":
        stack.tampilkan(mode="indeks", ascending=False)
    elif sub == "5":
        stack.tampilkan(mode="alphabet", ascending=True)
    elif sub == "6":
        stack.tampilkan(mode="alphabet", ascending=False)
    else:
        print("Pilihan tidak valid, menampilkan urutan tumpukan.")
        stack.tampilkan(mode="tumpukan")
```

Saat user memilih menu 3, program akan menampilkan sub-menu untuk memilih urutan tampilan barang. User dapat memilih untuk mengurutkan berdasarkan berat (dari ringan ke berat atau sebaliknya), indeks masuk (ascending atau descending), atau nama barang (A → Z atau Z → A). Program akan menampilkan daftar barang sesuai dengan urutan yang dipilih, beserta total berat dan kapasitas maksimum.

```
# =====
# MENU 4 -> JUMLAH BARANG
# =====
elif pilihan == "4":
    jumlah = stack.jumlah_barang()
    print(f"\nJumlah barang dalam stack: {jumlah}")
```

Saat user memilih menu 4, program akan menghitung dan menampilkan jumlah barang yang saat ini ada dalam stack. Jika stack kosong, jumlah yang ditampilkan akan menjadi 0.

```
# =====
# MENU 5 -> TOTAL BERAT
# =====
elif pilihan == "5":
    stack.total_berat()
```

Saat user memilih menu 5, program akan menghitung total berat seluruh barang yang ada dalam stack dan menampilkan informasi tersebut.

```
# =====
# MENU 6 -> UBAH INFORMASI BARANG
# =====
elif pilihan == "6":
    ubah_indeks = input("Masukkan indeks barang yang ingin diubah: ")
    nama_baru = input("Masukkan nama baru: ")
    try:
        berat_baru = float(input("Masukkan berat baru (kg): "))
        if berat_baru <= 0:
            print("Masukkan berat yang valid (lebih dari 0 kg)!")
        else:
            stack.ubah_informasi(ubah_indeks, nama_baru, berat_baru)
    except ValueError:
        print("Input berat harus berupa angka!")
```

Saat user memilih menu 6, program akan meminta indeks barang yang ingin diubah, nama baru, dan berat baru. Program akan mencari barang dengan indeks tersebut dalam stack. Jika ditemukan, informasi barang akan diperbarui dengan nama dan berat baru yang dimasukkan oleh user. Jika indeks tidak ditemukan atau input berat tidak valid, program akan memberikan pesan kesalahan yang sesuai.

```
# =====
# MENU 7 -> CARI BARANG
# =====
elif pilihan == "7":
    kata_kunci = input("Masukkan kata kunci pencarian: ")
    stack.cari(kata_kunci)
```

Saat user memilih menu 7, program akan meminta kata kunci untuk pencarian barang. Program akan mencari semua barang dalam stack yang nama barangnya mengandung kata kunci tersebut (pencarian case-insensitive). Jika ditemukan barang yang cocok, program akan menampilkan daftar barang tersebut beserta total berat dan kapasitas maksimum. Jika tidak ada barang yang cocok, program akan memberitahu bahwa tidak ada hasil yang ditemukan.

```
# =====
# MENU 0 -> KELUAR
# =====
elif pilihan == "0":
    print("\nTerima kasih telah menggunakan program.")
    print("Program selesai.")
    break
```

Saat user memilih menu 0, program akan menampilkan pesan terima kasih dan mengakhiri program dengan keluar dari loop utama.

```
# =====
# INPUT MENU SALAH
# =====
else:
    print("\nPilihan tidak valid!")
    print("Silakan pilih menu 0 - 7.")
```

Setelah setiap operasi, program akan kembali menampilkan menu utama untuk memungkinkan user melakukan operasi lain atau keluar dari program.

```
# =====
# MENJALANKAN PROGRAM
# =====
if __name__ == "__main__":
    main()
```

